

Add constexpr Modifiers to Functions in <algorithm> and <cstring> Headers

I. Introduction and Motivation

The Standard Library provides a great collection of containers and algorithms, many of which currently lack constexpr support. Even a simple constexpr usage requires reimplementing a big bunch of the Standard Library. Consider the simple example:

```
#include <array>
#include <algorithm>

int main() {
    // OK
    constexpr std::array<char, 6> a { 'H', 'e', 'l', 'l', 'o' };

    // Failures:
    // * std::find is not constexpr
    // * std::array::rbegin(), std::array::rend() are not constexpr
    // * std::array::reverse_iterator is not constexpr
    constexpr auto it = std::find(a.rbegin(), a.rend(), 'H');
}
```

This proposal concentrates on constexpr algorithms, deferring simple containers and iterators to a separate proposal.

A proof of concept implementation for some algorithms, is available at: [rhalbersma](#) and [Boost.Algorithm](#).

II. Impact on the Standard

This proposal is a pure library extension. It proposes changes to existing headers <cstring> and <algorithm> such that the changes do not break existing code and do not degrade performance. It does not require any changes in the core language in simple cases of non assembly optimized Standard Library, and it could be implemented in standard C++, except for the memcpy and memmove functions.

Depending on the Standard Library implementation this proposal may rely on [P0031R0](#). P0031R0 has been approved by LEWG and is on LWG's agenda for the next meeting. P0031R0 provides constexpr additions to std::advance, std::distance, std::move_iterator and other functions and classes. Those may be used by some implementations of <algorithm> header.

III. Design Decisions

A. <cstring> must have constexpr additions

Existing implementations of the functions in <algorithm> header usually rely on functions from <cstring>. For example std::copy usually takes advantage of std::memmove for POD types. This leads us to situation, that functions in <algorithm> header could not be marked as constexpr without constexpr marking functions from <cstring>.

B. std::memmove and std::memcpy must have constexpr additions

std::memmove and std::memcpy accept void* and const void* parameters. This makes them impossible to implement in pure C++ as constexpr, because constant expressions can not evaluate a conversion from type cv void* to a pointer-to-object type according to [expr.const].

However those functions are not only popular, but also are widely used across Standard Library to gain better performance. Not making them constexpr will force standard Library developer to have compiler intrinsics for them anyway. This is a hard step that must be done.

C. Assumption that it is possible to implement all the proposed changes without affecting language core, especially [expr.const]

There are many Standard Library implementations nowadays, including some proprietary. It is impossible to investigate all of them to be 100% sure that no performance degradation possible.

This proposal assumes that:

- If algorithm uses compiler intrinsics, then those intrinsics could be made constexpr by compiler vendors.
- If algorithm uses assembly optimization, then that assembly could be turned into constexpr compiler intrinsic.
- If algorithm uses external functions, then those functions could be made inline and marked constexpr or could be replaced with intrinsics.
- Modern compilers are good in code optimization, so a decently small amount of algorithms use assembly or intrinsics.

IV. Proposed wording relative to N4567

All the additions to the Standard are marked with underlined green.

A. Modifications to "21.8 Null-terminated sequence utilities" [c.strings]

Tables 73, 74, 75, 76, 77, and 78 describe headers <cctype>, <cwctype>, <cstring>, <wchar>, <cstdlib> (character conversions), and <uchar>, respectively.

The contents of these headers shall be the same as the Standard C Library headers <ctype.h>, <wctype.h>, <string.h>, <wchar.h>, and <stdlib.h> and the C Unicode TR header <uchar.h>, respectively, with the following modifications:

The headers shall not define the types char16_t, char32_t, and wchar_t (2.11).

All the functions from <cstring> header must be marked with constexpr, except the strcoll, strxfrm, strerror functions.

The function signature strchr(const char*, int) shall be replaced by the two declarations:

```
constexpr const char* strchr(const char* s, int c);
constexpr char* strchr(char* s, int c);
```

both of which shall have the same behavior as the original declaration.

The function signature strpbrk(const char*, const char*) shall be replaced by the two declarations:

```
constexpr const char* strpbrk(const char* s1, const char* s2);
constexpr char* strpbrk(char* s1, const char* s2);
both of which shall have the same behavior as the original declaration.
```

The function signature `strchr(const char*, int)` shall be replaced by the two declarations:

```
constexpr const char* strchr(const char* s, int c);
constexpr char* strchr(char* s, int c);
both of which shall have the same behavior as the original declaration.
```

The function signature `strstr(const char*, const char*)` shall be replaced by the two declarations:

```
constexpr const char* strstr(const char* s1, const char* s2);
constexpr char* strstr(char* s1, const char* s2);
both of which shall have the same behavior as the original declaration.
```

The function signature `memchr(const void*, int, size_t)` shall be replaced by the two declarations:

```
constexpr const void* memchr(const void* s, int c, size_t n);
constexpr void* memchr(void* s, int c, size_t n);
both of which shall have the same behavior as the original declaration.
```

B. Modifications to "Header <algorithm> synopsis" [algorithms.general]

```
#include <initializer_list>
```

```
namespace std {
```

```
// 25.2, non-modifying sequence operations:
```

```
template <class InputIterator, class Predicate>
constexpr bool all_of(InputIterator first, InputIterator last, Predicate pred);
```

```
template <class InputIterator, class Predicate>
constexpr bool any_of(InputIterator first, InputIterator last, Predicate pred);
```

```
template <class InputIterator, class Predicate>
constexpr bool none_of(InputIterator first, InputIterator last, Predicate pred);
```

```
template<class InputIterator, class Function>
constexpr Function for_each(InputIterator first, InputIterator last, Function f);
```

```
template<class InputIterator, class T>
constexpr InputIterator find(InputIterator first, InputIterator last, const T& value);
```

```
template<class InputIterator, class Predicate>
constexpr InputIterator find_if(InputIterator first, InputIterator last, Predicate pred);
```

```
template<class InputIterator, class Predicate>
constexpr InputIterator find_if_not(InputIterator first, InputIterator last, Predicate pred);
```

```
template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator1 find_end(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2);
```

```
template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
constexpr ForwardIterator1 find_end(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2, BinaryPredicate pred);
```

```
template<class InputIterator, class ForwardIterator>
constexpr InputIterator find_first_of(InputIterator first1, InputIterator last1, ForwardIterator first2, ForwardIterator last2);
```

```
template<class InputIterator, class ForwardIterator, class BinaryPredicate>
constexpr InputIterator find_first_of(InputIterator first1, InputIterator last1, ForwardIterator first2, ForwardIterator last2, BinaryPredicate pred);
```

```
template<class ForwardIterator>
constexpr ForwardIterator adjacent_find(ForwardIterator first, ForwardIterator last);
```

```
template<class ForwardIterator, class BinaryPredicate>
constexpr ForwardIterator adjacent_find(ForwardIterator first, ForwardIterator last, BinaryPredicate pred);
```

```
template<class InputIterator, class T>
constexpr typename iterator_traits<InputIterator>::difference_type count(InputIterator first, InputIterator last, const T& value);
```

```
template<class InputIterator, class Predicate>
constexpr typename iterator_traits<InputIterator>::difference_type count_if(InputIterator first, InputIterator last, Predicate pred);
```

```
template<class InputIterator1, class InputIterator2>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2);
```

```
template <class InputIterator1, class InputIterator2, class BinaryPredicate>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, BinaryPredicate pred);
```

```
template<class InputIterator1, class InputIterator2>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2);
```

```
template <class InputIterator1, class InputIterator2, class BinaryPredicate>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, BinaryPredicate pred);
```

```
template<class InputIterator1, class InputIterator2>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2);
```

```
template <class InputIterator1, class InputIterator2, class BinaryPredicate>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, BinaryPredicate pred);
```

```
template<class InputIterator1, class InputIterator2>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2);
```

```
template <class InputIterator1, class InputIterator2, class BinaryPredicate>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, BinaryPredicate pred);
```

```
template<class ForwardIterator1, class ForwardIterator2>
constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2);
```

```
template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, BinaryPredicate pred);
```

```
template<class ForwardIterator1, class ForwardIterator2>
constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2);
```

```

template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2, BinaryPredicate pred);

template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator1 search( ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2);

template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
constexpr ForwardIterator1 search(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2, ForwardIterator2 last2, BinaryPredicate pred);

template<class ForwardIterator, class Size, class T>
constexpr ForwardIterator search_n(ForwardIterator first, ForwardIterator last, Size count, const T& value);

template <class ForwardIterator, class Size, class T, class BinaryPredicate>
constexpr ForwardIterator search_n(ForwardIterator first, ForwardIterator last, Size count, const T& value, BinaryPredicate pred);

// 25.3, modifying sequence operations:
// 25.3.1, copy:
template<class InputIterator, class OutputIterator>
constexpr OutputIterator copy(InputIterator first, InputIterator last, OutputIterator result);

template<class InputIterator, class Size, class OutputIterator>
constexpr OutputIterator copy_n(InputIterator first, Size n, OutputIterator result);

template<class InputIterator, class OutputIterator, class Predicate>
constexpr OutputIterator copy_if(InputIterator first, InputIterator last, OutputIterator result, Predicate pred);

template<class BidirectionalIterator1, class BidirectionalIterator2>
constexpr BidirectionalIterator2 copy_backward(BidirectionalIterator1 first, BidirectionalIterator1 last, BidirectionalIterator2 result);

// 25.3.2, move:
template<class InputIterator, class OutputIterator>
constexpr OutputIterator move(InputIterator first, InputIterator last, OutputIterator result);

template<class BidirectionalIterator1, class BidirectionalIterator2>
constexpr BidirectionalIterator2 move_backward(BidirectionalIterator1 first, BidirectionalIterator1 last, BidirectionalIterator2 result);

// 25.3.3, swap:
template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator2 swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2);

template<class ForwardIterator1, class ForwardIterator2>
constexpr void iter_swap(ForwardIterator1 a, ForwardIterator2 b);

template<class InputIterator, class OutputIterator, class UnaryOperation>
constexpr OutputIterator transform(InputIterator first, InputIterator last, OutputIterator result, UnaryOperation op);

template<class InputIterator1, class InputIterator2, class OutputIterator, class BinaryOperation>
constexpr OutputIterator transform(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, OutputIterator result, BinaryOperation binary_op);

template<class ForwardIterator, class T>
constexpr void replace(ForwardIterator first, ForwardIterator last, const T& old_value, const T& new_value);

template<class ForwardIterator, class Predicate, class T>
constexpr void replace_if(ForwardIterator first, ForwardIterator last, Predicate pred, const T& new_value);

template<class InputIterator, class OutputIterator, class T>
constexpr OutputIterator replace_copy(InputIterator first, InputIterator last, OutputIterator result, const T& old_value, const T& new_value);

template<class InputIterator, class OutputIterator, class Predicate, class T>
constexpr OutputIterator replace_copy_if(InputIterator first, InputIterator last, OutputIterator result, Predicate pred, const T& new_value);

template<class ForwardIterator, class T>
constexpr void fill(ForwardIterator first, ForwardIterator last, const T& value);

template<class OutputIterator, class Size, class T>
constexpr OutputIterator fill_n(OutputIterator first, Size n, const T& value);

template<class ForwardIterator, class Generator>
constexpr void generate(ForwardIterator first, ForwardIterator last, Generator gen);

template<class OutputIterator, class Size, class Generator>
constexpr OutputIterator generate_n(OutputIterator first, Size n, Generator gen);

template<class ForwardIterator, class T>
constexpr ForwardIterator remove(ForwardIterator first, ForwardIterator last, const T& value);

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator remove_if(ForwardIterator first, ForwardIterator last, Predicate pred);

template<class InputIterator, class OutputIterator, class T>
constexpr OutputIterator remove_copy(InputIterator first, InputIterator last, OutputIterator result, const T& value);

template<class InputIterator, class OutputIterator, class Predicate>
constexpr OutputIterator remove_copy_if(InputIterator first, InputIterator last, OutputIterator result, Predicate pred);

template<class ForwardIterator>
constexpr ForwardIterator unique(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class BinaryPredicate>
constexpr ForwardIterator unique(ForwardIterator first, ForwardIterator last, BinaryPredicate pred);

template<class InputIterator, class OutputIterator>
constexpr OutputIterator unique_copy(InputIterator first, InputIterator last, OutputIterator result);

template<class InputIterator, class OutputIterator, class BinaryPredicate>
constexpr OutputIterator unique_copy(InputIterator first, InputIterator last, OutputIterator result, BinaryPredicate pred);

template<class BidirectionalIterator>
constexpr void reverse(BidirectionalIterator first, BidirectionalIterator last);

template<class BidirectionalIterator, class OutputIterator>
constexpr OutputIterator reverse_copy(BidirectionalIterator first, BidirectionalIterator last, OutputIterator result);

template<class ForwardIterator>
constexpr ForwardIterator rotate(ForwardIterator first, ForwardIterator middle, ForwardIterator last);

template<class ForwardIterator, class OutputIterator>
constexpr OutputIterator rotate_copy(ForwardIterator first, ForwardIterator middle, ForwardIterator last, OutputIterator result);

// 25.3.12, shuffle:

```

```

template<class RandomAccessIterator, class UniformRandomNumberGenerator>
constexpr void shuffle(RandomAccessIterator first, RandomAccessIterator last, UniformRandomNumberGenerator&& g);

// 25.3.13, partitions:
template <class InputIterator, class Predicate>
constexpr bool is_partitioned(InputIterator first, InputIterator last, Predicate pred);

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator partition(ForwardIterator first, ForwardIterator last, Predicate pred);

template<class BidirectionalIterator, class Predicate>
constexpr BidirectionalIterator stable_partition(BidirectionalIterator first, BidirectionalIterator last, Predicate pred);

template <class InputIterator, class OutputIterator1, class OutputIterator2, class Predicate>
constexpr pair<OutputIterator1, OutputIterator2>
    partition_copy(InputIterator first, InputIterator last, OutputIterator1 out_true, OutputIterator2 out_false, Predicate pred);

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator partition_point(ForwardIterator first, ForwardIterator last, Predicate pred);

// 25.4, sorting and related operations:
// 25.4.1, sorting:
template<class RandomAccessIterator>
constexpr void sort(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void sort(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr void stable_sort(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void stable_sort(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr void partial_sort(RandomAccessIterator first, RandomAccessIterator middle, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void partial_sort(RandomAccessIterator first, RandomAccessIterator middle, RandomAccessIterator last, Compare comp);

template<class InputIterator, class RandomAccessIterator>
constexpr RandomAccessIterator partial_sort_copy(InputIterator first, InputIterator last, RandomAccessIterator result_first, RandomAccessIterator result_last);

template<class InputIterator, class RandomAccessIterator, class Compare>
constexpr RandomAccessIterator partial_sort_copy(InputIterator first, InputIterator last, RandomAccessIterator result_first, RandomAccessIterator result_last, Compare comp);

template<class ForwardIterator>
constexpr bool is_sorted(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
constexpr bool is_sorted(ForwardIterator first, ForwardIterator last, Compare comp);

template<class ForwardIterator>
constexpr ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
constexpr ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth, RandomAccessIterator last, Compare comp);

// 25.4.3, binary search:
template<class ForwardIterator, class T>
constexpr ForwardIterator lower_bound(ForwardIterator first, ForwardIterator last, const T& value);

template<class ForwardIterator, class T, class Compare>
constexpr ForwardIterator lower_bound(ForwardIterator first, ForwardIterator last, const T& value, Compare comp);

template<class ForwardIterator, class T>
constexpr ForwardIterator upper_bound(ForwardIterator first, ForwardIterator last, const T& value);

template<class ForwardIterator, class T, class Compare>
constexpr ForwardIterator upper_bound(ForwardIterator first, ForwardIterator last, const T& value, Compare comp);

template<class ForwardIterator, class T>
constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last, const T& value);

template<class ForwardIterator, class T, class Compare>
constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last, const T& value, Compare comp);

template<class ForwardIterator, class T>
constexpr bool binary_search(ForwardIterator first, ForwardIterator last, const T& value);

template<class ForwardIterator, class T, class Compare>
constexpr bool binary_search(ForwardIterator first, ForwardIterator last, const T& value, Compare comp);

// 25.4.4, merge:
template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator merge(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result);

template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator merge(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result, Compare comp);

template<class BidirectionalIterator>
constexpr void inplace_merge(BidirectionalIterator first, BidirectionalIterator middle, BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
constexpr void inplace_merge(BidirectionalIterator first, BidirectionalIterator middle, BidirectionalIterator last, Compare comp);

// 25.4.5, set operations:
template<class InputIterator1, class InputIterator2>
constexpr bool includes(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2);

template<class InputIterator1, class InputIterator2, class Compare>

```

```
constexpr bool includes(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, Compare comp);

template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator set_union(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result);

template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator set_union(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result, Compare comp);

template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator set_intersection(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result);

template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator set_intersection(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result, Compare comp);

template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator set_difference(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result);

template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator set_difference(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result, Compare comp);

template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator set_symmetric_difference(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result);

template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator set_symmetric_difference(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, OutputIterator result, Compare comp);

// 25.4.6, heap operations:
template<class RandomAccessIterator>
constexpr void push_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void push_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr void pop_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void pop_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr void make_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void make_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr void sort_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void sort_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr bool is_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr bool is_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

template<class RandomAccessIterator>
constexpr RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last, Compare comp);

// 25.4.7, minimum and maximum:
template<class T> constexpr const T& min(const T& a, const T& b);

template<class T, class Compare>
constexpr const T& min(const T& a, const T& b, Compare comp);

template<class T>
constexpr T min(initializer_list<T> t);

template<class T, class Compare>
constexpr T min(initializer_list<T> t, Compare comp);

template<class T> constexpr const T& max(const T& a, const T& b);

template<class T, class Compare>
constexpr const T& max(const T& a, const T& b, Compare comp);

template<class T>
constexpr T max(initializer_list<T> t);

template<class T, class Compare>
constexpr T max(initializer_list<T> t, Compare comp);

template<class T> constexpr pair<const T&, const T&> minmax(const T& a, const T& b);

template<class T, class Compare>
constexpr pair<const T&, const T&> minmax(const T& a, const T& b, Compare comp);

template<class T>
constexpr pair<T, T> minmax(initializer_list<T> t);

template<class T, class Compare>
constexpr pair<T, T> minmax(initializer_list<T> t, Compare comp);

template<class ForwardIterator>
constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last)

template<class ForwardIterator, class Compare>
constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last, Compare comp);

template<class ForwardIterator>
constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last)

template<class ForwardIterator, class Compare>
constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last, Compare comp);

template<class ForwardIterator>
```

```
constexpr pair<ForwardIterator, ForwardIterator>
    minmax_element(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
constexpr pair<ForwardIterator, ForwardIterator>
    minmax_element(ForwardIterator first, ForwardIterator last, Compare comp);

template<class InputIterator1, class InputIterator2>
constexpr bool lexicographical_compare(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2);

template<class InputIterator1, class InputIterator2, class Compare>
constexpr bool lexicographical_compare(InputIterator1 first1, InputIterator1 last1, InputIterator2 first2, InputIterator2 last2, Compare comp);

// 25.4.9, permutations:
template<class BidirectionalIterator>
constexpr bool next_permutation(BidirectionalIterator first, BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
constexpr bool next_permutation(BidirectionalIterator first, BidirectionalIterator last, Compare comp);

template<class BidirectionalIterator>
constexpr bool prev_permutation(BidirectionalIterator first, BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
constexpr bool prev_permutation(BidirectionalIterator first, BidirectionalIterator last, Compare comp);

}
```

C. Modifications to remaining parts of "25 Algorithms library" [algorithms] (all sections except "Header <algorithm> synopsis" and "25.5 C library algorithms" [alg.c.library])

Note for editor: All the functions marked with `constexpr` in previous paragraph of this document must be accordingly marked with `constexpr` in detailed algorithm description. For shortness only modifications to "25.2.1 All of" [alg.all_of] are shown in this paper.

25.2.1 All of [alg.all_of]

```
template <class InputIterator, class Predicate>
constexpr bool all_of(InputIterator first, InputIterator last, Predicate pred);

Returns: true if [first,last) is empty or if pred(*i) is true for every iterator i in the range
[first,last), and false otherwise.

Complexity: At most last - first applications of the predicate.
```

D. Modifications to "20.2 Utility components" [utility]

```
// 20.2.2, swap:
template<class T>
constexpr void swap(T& a, T& b) noexcept(see below );

template <class T, size_t N>
constexpr void swap(T (&a)[N], T (&b)[N]) noexcept(noexcept(swap(*a, *b)));

// 20.2.3, exchange:
template <class T, class U=T>
constexpr T exchange(T& obj, U&& new_val);
```

E. Modifications to "20.2.2 swap" [utility.swap]

```
template<class T> constexpr void swap(T& a, T& b) noexcept(see below );

Remark: The expression inside noexcept is equivalent to:
    is_nothrow_move_constructible<T>::value &&
    is_nothrow_move_assignable<T>::value

Requires: Type T shall be MoveConstructible (Table 20) and MoveAssignable (Table 22).
Effects: Exchanges values stored in two locations.

template<class T, size_t N>
constexpr void swap(T (&a)[N], T (&b)[N]) noexcept(noexcept(swap(*a, *b)));

Requires: a[i] shall be swappable with (17.6.3.2) b[i] for all i in the range [0,N).
Effects: swap_ranges(a, a + N, b)
```

F. Modifications to "20.2.3 exchange" [utility.exchange]

```
template <class T, class U=T> constexpr T exchange(T& obj, U&& new_val);

Effects: Equivalent to:
    T old_val = std::move(obj);
    obj = std::forward<U>(new_val);
    return old_val;
```

G. Feature-testing macro

For the purposes of SG10, we recommend the feature-testing macro name `__cpp_lib_constexpr_algorithms`.

V. Revision History

- Revision 0:
- Initial proposal

VI. References

[N4567] Working Draft, Standard for Programming Language C++. Available online at www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4567.pdf

[P0031R0] A Proposal to Add Constexpr Modifiers to reverse_iterator, move_iterator, array and Range Access. Available online at www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0031r0.pdf

std.org/jtc1/sc22/wg21/docs/papers/2015/p0031r0.html

[rhalbersma] Proof of concept for some functions. Available online at <https://bitbucket.org/rhalbersma/xstd/src/42553df6107623c71163f104b6f3cc550c245b4b/include/xstd/algorithm.hpp> and <https://bitbucket.org/rhalbersma/xstd/src/42553df6107623c71163f104b6f3cc550c245b4b/include/xstd/utility.hpp>

[Boost.Algorithm] Constexpr modifiers for Boost Algorithm library. Available online at <https://github.com/boostorg/algorithm/pull/13>

[Discussion] A call to discuss asm in constexpr and constexpr <algorithm>. Available online at <https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/9sTJWsOpptE>

VII. Acknowledgements

Walter E. Brown provided numerous comments, corrections, and suggestions for this proposal.